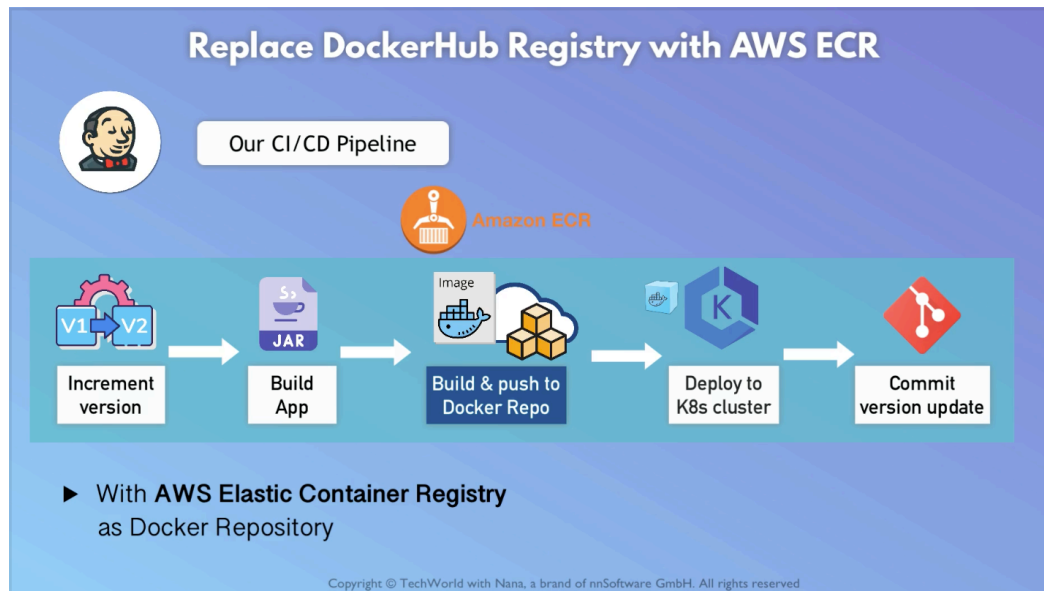


Module 11 - chapter 10

Complete CI/CD Pipeline with EKS and ECR

Now we will use AWS ECR instead of dockerhub



We need to create

- an ECR repository

Create ECR Repository

- Unlimited number of private repositories
- Repository per application

- ECR Credentials in Jenkins
- Create a secret -> will fetch the image from ECR via a secret

Steps we need to do

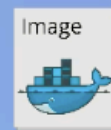
Create ECR Repository



Create Credentials in Jenkins



Adjust building and tagging



Create Secret for ECR



Update Jenkinsfile



We need to edit the Jenkinsfile:

- 'Build image' stage -> needs to point at ECR instead of dockerhub and for that we use the new Jenkins credentials for ECR

First I need to create a new EKS cluster

```
-> eksctl create cluster \  
    --name demo-cluster \  
    --version 1.35 \  
    --region eu-west-2 \  
    --nodegroup-name demo-nodes \  
    --node-type t2.micro \  
    --nodes 2 \  
    --nodes-min 1 \  
    --nodes-max 3
```

And edit the config file in my droplet, so Jenkins container gets it as well:

```

apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ01JU0hZa
    UTUJFR0ExVUUKQXhNS2EzVm1aWEp1WlhSbGN6QWVGdzB5TmBmk1USXdPRE01TWpkYUz3Mhp0akEyTURrd09EUTBNakphTUJVeApF
    1BMEdDU3FHU01iM0RRRUJBUVVBQTRJQkR3QXdnZ0VLCkFvSUJBUURCRHRSZdCNENkZEFRL3h4R01weVliQVdIODNBNOpPdDBCSmF
    Wx6dzU4d3E4U0hRa2QvYWNyRXg3a1FmakkrRExZWndxRkFLZkFmN09hczRSM0xhZTF0UDlrMapXbmJYdGhjtJluYjM0eVxx0TNIT3
    ODZuSXJaek03K0h1CnFyOVRDMLVFfa2EwU1UzWk5UcUxpTVFhYVNSQ2N2cXZNNnA4cFllRkVmK3hNcVRXSDlRQU1HRE1BU1l4RmxxpW
    sb1hZR2ZGMHRJY2RKR21teWRoN1hET01ocHozafA0ekIyZwpzUkQrbEowbHJRRnM1Q2ZSMVlIT25Wai50G5qQWdNQkFBR2pXVEJY
    1CQWY4RUJQUURBUUgVtUIIwR0ExVWREZ1FXQkJSSTEhaWJc2V1lk0E9HVEw2M2l0OG0vMmI4aTZUQVYKQmdOVkhSRUVEakFNZ2dwcmlR
    TRJQkFRQnBpU0o2TzFUMApOT0J6bUxScE5ubE840TV0U1pWk2hEcThRclMxSHdvczhjNDgzNjR3Z29Cejh4TGhZeDhSTEVBWbZFLUm
    bUtVa1BCYWxLU1cvb2VKR1U1Q1c0SEU4SittGdk1yRnZ1Q08yUHoKe1RnL21xUFFnV21UMGN6Z0lvSjMvd0plQzhBeGJiK1hIZGtwZ
    aSHUwSHdqT0M0b21yL09EdVZPTThReTZVYkFDNlJWMMUJXbDNMc3F4c3Q3ZDAyU2ExY1VuL2FrakQ3eW4rCkFvQVE0UWN1cGpYVXQ4
    RKTXRNTUdzNGo0WHRCdFNYWEQKQWZlYXZTLV3S1FBCi0tLS0tRU5EIENFU1RJRklDQVRFLS0tLS0K
    server: https://354DF61B43547A65FB2BBD156F4812F7.gr7.eu-west-2.eks.amazonaws.com
name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - "demo-cluster"

```

And update aws-auth -> Because I created a new EKS cluster — the aws-auth ConfigMap starts fresh with only the node group role.

```

mapUsers: |
- userarn: arn:aws:iam::706976333940:user/jenkins
  username: jenkins
  groups:
  - system:masters

```

-> `kubectl edit configmap aws-auth -n kube-system`

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  mapRoles: |
    - rolearn: arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-demo-NodeInstanceRole-fj7BftVan9DA
      groups:
        - system:bootstrappers
        - system:nodes
      username: system:node:{{EC2PrivateDNSName}}
  mapUsers: |
    - userarn: arn:aws:iam::706976333940:user/jenkins
      username: jenkins
      groups:
        - system:masters
kind: ConfigMap
metadata:
  creationTimestamp: "2026-06-12T08:51:22Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "1259"
  uid: 396a8e86-a4d9-463e-b2c2-f0b292bd7596
```

```
astrid@192.168.1.112 /Users/astrid
% kubectl edit configmap aws-auth -n kube-system
configmap/aws-auth edited
astrid@192.168.1.112 /Users/astrid
```

Let's start!

Create ECR Repository

I give it the name of my app -> java-maven-app

Amazon ECR > Private registry > Repositories > Create a private repository

General settings

Repository name
Enter a concise name. Repositories support namespaces, which you can use to group similar repositories.

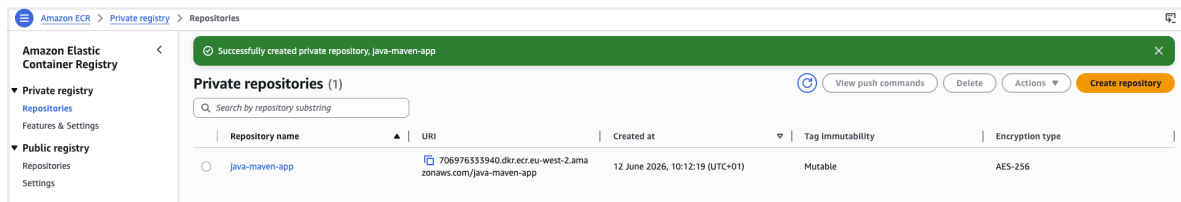
706976333940.dkr.ecr.eu-west-2.amazonaws.com/

14 out of 256 characters maximum (2 minimum). The name must start with a letter and can only contain lowercase letters, numbers and special characters.

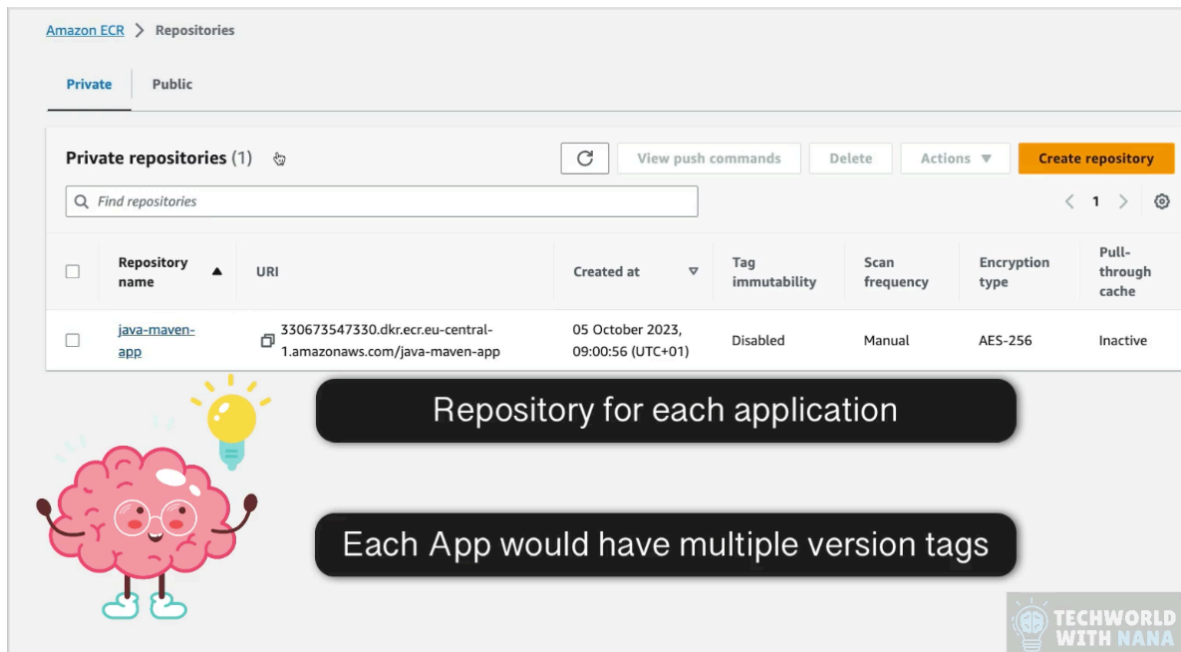
The whole repository name is -> 706976333940.dkr.ecr.eu-west-2.amazonaws.com/java-maven-app

706976333940 -> my AWS account

And it is created:

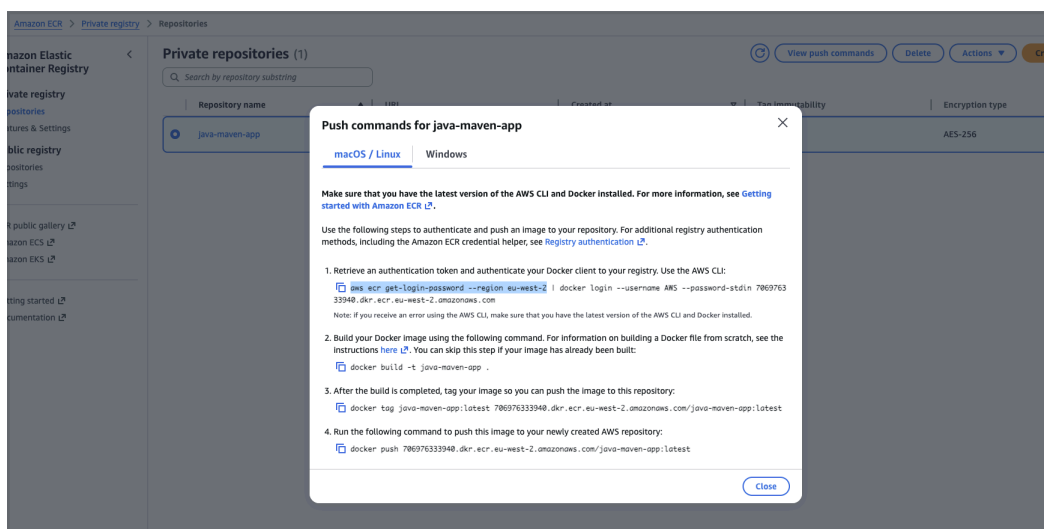


Each app or each microservice will have their own repo, so in this case all my java-maven-app version (version tags) will be in the repo java-maven-app.



Create Credentials in Jenkins

Click on view Push commands:



It tells us how to get an authentication token:

-> `aws ecr get-login-password --region eu-west-2 | docker login --username AWS --password-stdin 706976333940.dkr.ecr.eu-west-2.amazonaws.com`

First part:

I run this part -> `aws ecr get-login-password --region eu-west-2`

And I get the ECR token -> NOTE: The password lasts 12hrs since creation, after that is not valid any longer.

```
astrid@192.168.1.112 /Users/astrid
% aws ecr get-login-password --region eu-west-2
eyJwYXlsb2FkIjoiczZXV2lLRkpNTkptUkdzRDNnSmo2em5tb3JNVlJwTXpwNG5Ic01VT
zM293U0Nxa1ZxQjFUVnZCSjQ4aDN3b1FScFd1RWJiRzF0Z3prRnda0UJ5LzVTaDFVdzBR
c0RzJHswXQZ2kxalFRXQ1RFRxZzJrKzRB0FAwNHRqdE5mUnhFQ0NzVGUzVEh6ZGF3TGR
zJ5eDJMZnhQemhzNDdsd002RnJLMjRVeUZKczNqZ1pmeC9yc1JWZzQ2dm1wUk1ZdUgva
TnkwNlNVVlBJMmVDdHU0Y2dIVTh2Z01aSDRQK2F6dmNNb1VDN1lQL1c5eWRlbzRHdE5mS
pM1ErMHh4NCtsUTlhL2FJYWNRNTNhYm0c3Ym84ZkpLMU15MndjVWpCa1hpemdjQ2tXWLZh
NTUWhnS1ErQk1lCFZhM0ZXRUhMK2RMeGNZL0c3VGptV2pWR3l3SlBVUWdUNW9VVTlyZDN
GphcHl3Mm1yZk9YT3dhQkhkZnVoZVhMU1dtTWVXMgXwR3lwak1zaThINHlWUGkzOVVtRV
RFhJQkt0enBDR1Z0VktRY1FTSVVCbk1Qb3VUcytEaXhBMUpuVHNUaFhSOTNhVWZQcDR4Q
3NF1KWGJNUFVZNC9KdTRpR2k5Vl1LaXNiY2w0SzJGTUQ2Q1RTajBoZEQzUGRQMy9FcHZQ
1yb1F6YTU3NFc0Y0hZTVU4MnRiTk13QUFBSDR3ZkFZSk1vWklodmNOQVFjR29H0HdiUUI
EVKZ0lCRU1BN1huVGRibUwWbDhqMytVM3di dVEyTHhsSXNJTSpnaEU5TU50cFZYaHhIZ1
eXB1IjoireFUQV9LRVkiLCJleHBpcmF0aW9uIjoxNzgxMjk5NzE4fQ==
astrid@192.168.1.112 /Users/astrid
% █
```

Second part:

-> `docker login --username AWS --password-stdin`

This is to login to docker and the username is AWS. The password is stdin (it is provided by the first part: `aws ecr get-login-password --region eu-west-2`)

Third part:

-> `706976333940.dkr.ecr.eu-west-2.amazonaws.com`

This is the docker repository URL, the ECR URL.

This information is part of the Jenkins stage 'build image'

```

e('build image') {
  steps {
    script {
      echo "building the docker image..."
      withCredentials([usernamePassword(credentialsId: 'docker-hub-repo', passwordVariable: 'PASS', usernameVariable: 'USER')]) {
        sh "docker build -t nanatwn/demo-app:${IMAGE_NAME} ."
        sh 'echo $PASS | docker login -u $USER --password-stdin'
        sh "docker push nanatwn/demo-app:${IMAGE_NAME}"
      }
    }
  }
}

e('deploy') {
  environment {
    AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
  }
}

```

docker.io = default

In Nana's demo from previous chapter, she didn't include the URL because when we don't provide it by default it is assumed that is DockerHub. We were pushing to DockerHub so she didn't write DockerHub's URL ([docker.io](https://hub.docker.com/)) as it is used by default.

However, in my branch I was using an env var called '*DOCKER_REPO_SERVER*' so in my case I did specify:

```

33     }
34     stage('build image') {
35       steps {
36         script {
37           echo "building the docker image..."
38           withCredentials([usernamePassword(credentialsId: 'docker-hub-repo', passwordVariable: 'PASS', usernameVariable: 'USER')]) {
39             sh "docker build -t ${DOCKER_REPO}:${IMAGE_NAME} ."
40             sh 'echo $PASS | docker login -u $USER --password-stdin ${DOCKER_REPO_SERVER}' Janashia, 05/10/2020
41             sh "docker push ${DOCKER_REPO}:${IMAGE_NAME}"
42           }
43         }
44       }
45     }

```

```

6       maven 'maven-3.9'
7     }
8     environment {
9       DOCKER_REPO_SERVER = 'docker.io' Pardo, Yesterday
10      DOCKER_REPO = 'cabfish00/demo-app'
11    }
12    stages {

```

As I already have the env var then all I need is to edit its value with the ECR URL.

For now let's add the new credential to Jenkins:

I create a new GLOBAL credential called 'ecr-credentials' and the name is AWS (remember the second part: docker login --username AWS provided in the view Push commands) and the password is the one created in my terminal

Created:

Create a Secret for AWS ECR

Jenkinsfile also triggers to pull the images for the 'deploy' stage. So we need a k8s secret to hold the credentials as well. In a similar way we did it for dockerhub when we created 'my-registry-key' locally in my laptop.

As I deleted the EKS cluster I don't have the secret 'my-registry-key' any longer

-> kubectl get secret

```
astrid@192.168.1.112 /Users/astrid
% kubectl get secret
No resources found in default namespace.
astrid@192.168.1.112 /Users/astrid
% █
```

This is the screenshot I got when I created 'my-registry-key' for Module 11 - chapter 9 EKS:


```

astrid@192.168.1.112 /Users/astrid
% secret/my-registry-key created

[1] + done      kubectl create secret docker-registry my-registry-key --docker-password=MpU
astrid@192.168.1.112 /Users/astrid
% kubectl get secret
NAME                TYPE                      DATA  AGE
my-registry-key     kubernetes.io/dockerconfigjson  1      56s
astrid@192.168.1.112 /Users/astrid
% █

```

Create ECR secret locally called 'aws-registry-key':

```

-> kubectl create secret docker-registry aws-registry-key \
--docker-server=706976333940.dkr.ecr.eu-west-2.amazonaws.com \
--docker-username=AWS \
--docker-password=*****

```

```

astrid@192.168.1.112 /Users/astrid
% kubectl create secret docker-registry aws-registry-key \
--docker-server=706976333940.dkr.ecr.eu-west-2.amazonaws.com \
--docker-username=AWS \
--docker-password=eyJwYXlsb2FkIjoiczZXV2lLRkpNTkptUkdzRDNnSmo2em5tb3JNVlJwTXp
0J0YjY4U3hkV1FMZjYzM293U0Nxa1ZxQjFUVnZCSjQ4aDN3b1FSY2F0Z3prRnda0UJ5Lz
dHk3aVFIQjhkQlAxOGc0RzJHSWxQZ2kxalFRXQ1RFRxZzJrKzRB0FAwNHRqdE5mUnhFQ0NzVGUzV
VbURjOGp4QjhTajYvdzJ5eDJMZhQemhZNDdsd002RnJLMjRveUZKczNqZ1pmeC9yc1JWZzQ2dm1w
Z2SkVQ1Z3cGV0YXNDTnkwNlNVVlBJMmVDdHU0Y2dIVTh2Z0laSDRQK2F6dmNNblVDN1lQL1c5eWR
zJlIaGg4aU0t1a1B1RWlpM1ErMHh4NCtsUTlhL2FJYWNRThhYm0c3Ym84ZkpLMU15MndjVWpCa1hpem
WlhnQmNUZ2xWYVERMGNTUWhnS1ErQk1lcFZhM0ZXRUhmK2RMeGNZL0c3VGptV2pWR3l3SlBVUWdUN
RWlRLV1BKRz1RNkhGaGphcHl3Mm1yZk9YT3dhQkhkZnVoZVhMU1dtTWVXMgxr3lwak1zaThINHlW
AvU0RETWd6cDFwMTZwRFhJQkt0enBDR1Z0VktRY1FTSVVCbk1Qb3VUcytEaXhBMUpuVHNUaFhSOTN
mFzK1hiV1pQekU4cWk3NF1KWGJNUFVZNC9KdTRpR2k5V1lLaXNiY2w0SzJGTUQ2Q1RTajBoZEQzUG
R3lsSzN0VWFsSEF0cG1yb1F6YTV3NFc0Y0hZTVU4MnRiTk13QUFBSDR3ZkFZSkvWklodmNOQVfJR
ITkNPVE15Q1hxTjI2eEVKZ0lCRU1BN1huVGRibUwwbDhQMytvM3didVEyTHhsSXNJTXpnaEU5TU50
Vyc2l2b2IiIjIiLCJ0eXB1IjoieFV0V9LRVkiLCJleHBpcmF0aW9uIjoxNzgxMjk5NzE4fQ==
secret/aws-registry-key created
astrid@192.168.1.112 /Users/astrid
% █

```

```

-> kubectl get secret

```

```

astrid@192.168.1.112 /Users/astrid
% kubectl get secret
NAME                TYPE                      DATA  AGE
aws-registry-key     kubernetes.io/dockerconfigjson  1      6m43s
astrid@192.168.1.112 /Users/astrid
% █

```

Now we can update the deployment.yaml file:

```

Jenkinsfile x deployment.yaml x README.md x service.yaml x Dockerfile x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: $APP_NAME
5    labels:
6      app: $APP_NAME
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: $APP_NAME
12    template:
13     metadata:
14       labels:
15         app: $APP_NAME
16     spec:
17       imagePullSecrets:
18         - name: aws-registry-key  You, Moments ago • Uncommitted changes
19     containers:
20       - name: $APP_NAME
21         image: $DOCKER_REPO:$IMAGE_NAME
22         imagePullPolicy: Always
23         ports:
24           - containerPort: 8080
25

```

Update Jenkinsfile

Change the credentials Id to use 'ecr-credentials' stored in Jenkins:

```

34  stage('build image') {
35    steps {
36      script {
37        echo "building the docker image..."
38        withCredentials([usernamePassword(credentialsId: 'ecr-credentials', passwordVariable: 'PASS', usernameVariable: 'USER')]) {
39          sh "docker build -t ${DOCKER_REPO}:${IMAGE_NAME} ."
40          sh 'echo $PASS | docker login -u $USER --password-stdin ${DOCKER_REPO_SERVER}'
41          sh "docker push ${DOCKER_REPO}:${IMAGE_NAME}"
42        }

```

Take the repo URL -> 706976333940.dkr.ecr.eu-west-2.amazonaws.com/java-maven-app

Private registry > Repositories		
Private repositories (1)		
Search by repository substring		
Repository name	Repository URI copied	Created at
java-maven-app	706976333940.dkr.ecr.eu-west-2.amazonaws.com/java-maven-app	12 June 2026, 10:12:19 (UTC+0)

And change the env value for `DOCKER_REPO_SERVER` and `DOCKER_REPO`
`DOCKER_REPO_SERVER = '706976333940.dkr.ecr.eu-west-2.amazonaws.com'`

```
DOCKER_REPO = "${DOCKER_REPO_SERVER}/java-maven-app"
```

```
8     environment {
9         DOCKER_REPO_SERVER = '706976333940.dkr.ecr.eu-west-2.amazonaws.com'
10        DOCKER_REPO = "${DOCKER_REPO_SERVER}/java-maven-app"
11    }
12    stages {
```

And they will be used in the 'build image' stage:

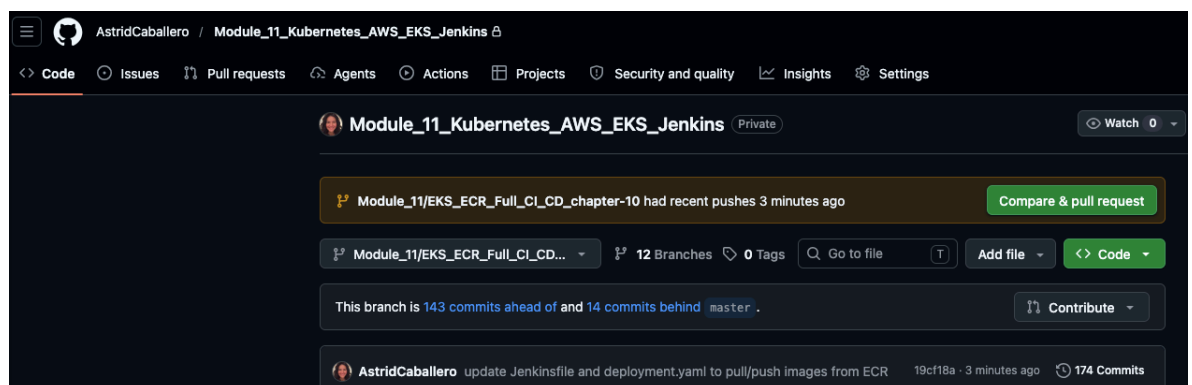
```
33    }
34    stage('build image') {
35        steps {
36            script {
37                echo "building the docker image..."
38                withCredentials([usernamePassword(credentialsId: 'ecr-credentials', passwordVariable: 'PASS', usernameVariable: 'USER')]) {
39                    sh "docker build -t ${DOCKER_REPO}:${IMAGE_NAME} ."
40                    sh 'echo $PASS | docker login -u $USER --password-stdin ${DOCKER_REPO_SERVER}'
41                    sh "docker push ${DOCKER_REPO}:${IMAGE_NAME}"
42                }
43            }
44        }
45    }
```

And the deployment.yaml file as well:

```
18     - name: aws-registry-key
19       containers:
20         - name: $APP_NAME
21           image: ${DOCKER_REPO}:${IMAGE_NAME}
22           imagePullPolicy: Always
23           ports:
24             - containerPort: 8080
25
```

Execute Jenkins Pipeline

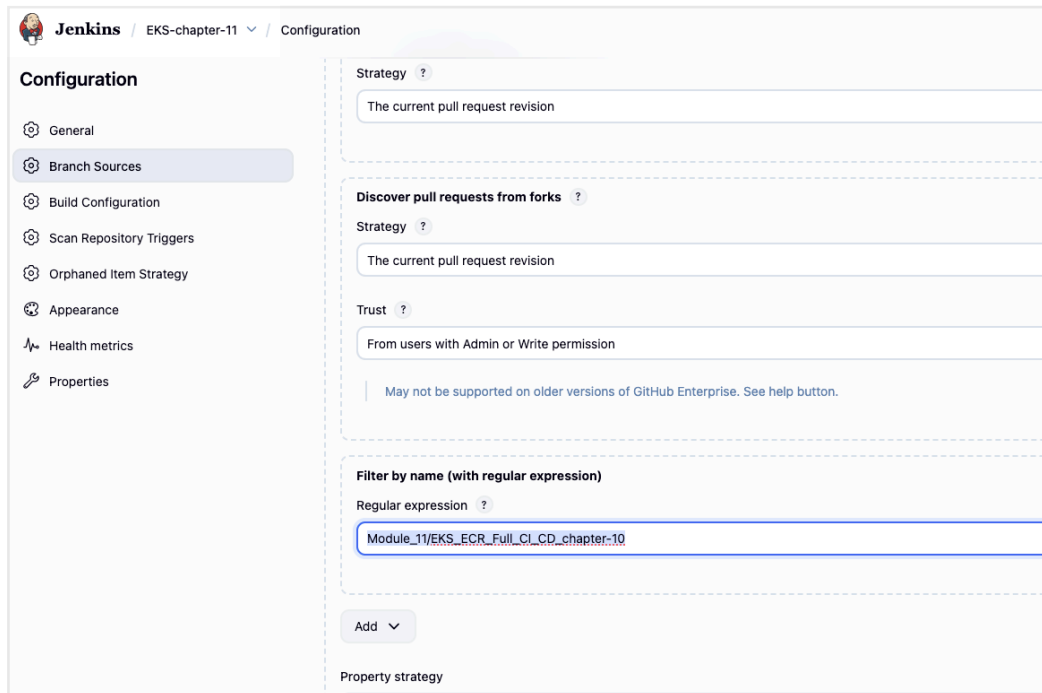
Push changes to GitHub:



The screenshot shows the GitHub interface for the repository 'Module_11_Kubernetes_AWS_EKS_Jenkins' by user 'AstridCaballero'. The repository is private. The main content area shows a recent push to the 'Module_11/EKS_ECR_Full_CI_CD_chapter-10' branch, which occurred 3 minutes ago. Below this, there are statistics for the repository: 12 branches, 0 tags, and a comparison showing the current branch is 143 commits ahead of and 14 commits behind the 'master' branch. At the bottom, a commit by 'AstridCaballero' is visible, titled 'update Jenkinsfile and deployment.yaml to pull/push images from ECR', made 3 minutes ago, with 174 total commits.

Edit pipeline to fetch new branch -> Module_11/

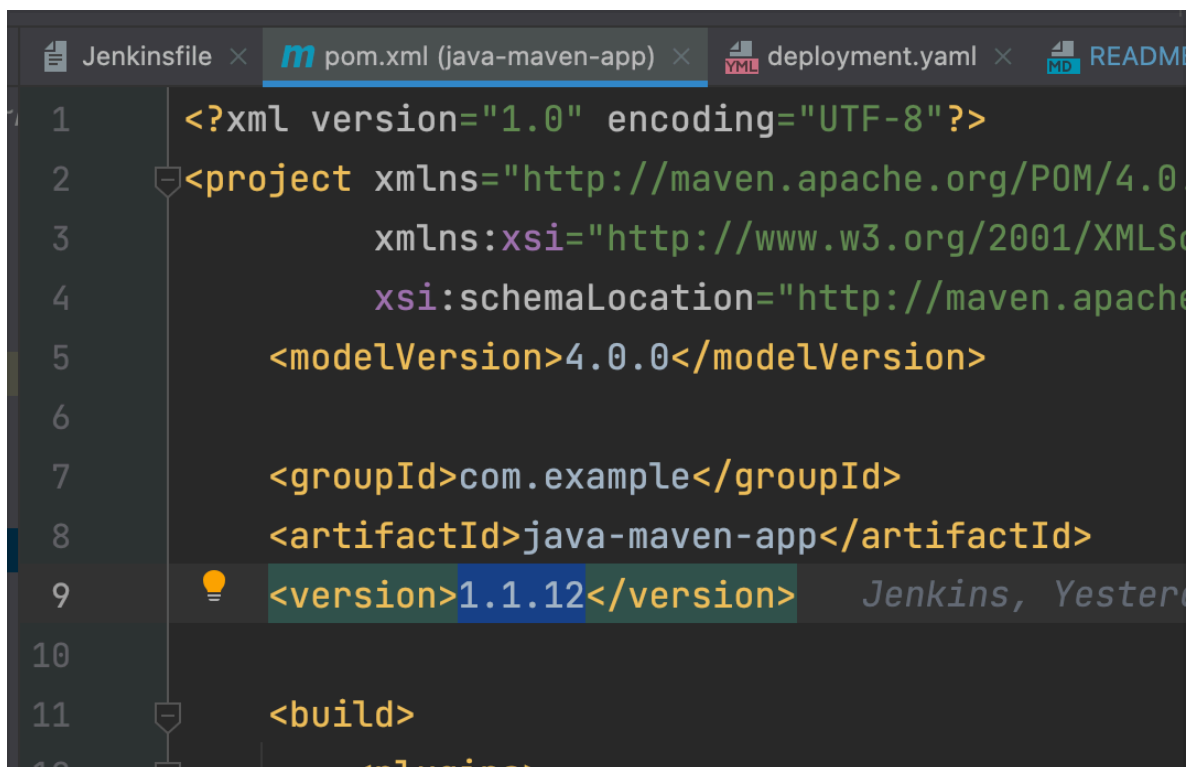
EKS_ECR_Full_CI_CD_chapter-10



The image shows the Jenkins Configuration page for the 'Branch Sources' section. The left sidebar contains a menu with options: General, Branch Sources (selected), Build Configuration, Scan Repository Triggers, Orphaned Item Strategy, Appearance, Health metrics, and Properties. The main content area is titled 'Configuration' and contains several sections:

- Strategy**: A dropdown menu with the selected option 'The current pull request revision'.
- Discover pull requests from forks**: A section with a 'Strategy' dropdown menu, also set to 'The current pull request revision'.
- Trust**: A dropdown menu with the selected option 'From users with Admin or Write permission'. Below it, a note states: 'May not be supported on older versions of GitHub Enterprise. See help button.'
- Filter by name (with regular expression)**: A section with a 'Regular expression' input field containing the text 'Module_11/EKS_ECR_Full_CI_CD_chapter-10'.
- Add**: A button with a dropdown arrow.
- Property strategy**: A label at the bottom of the configuration area.

Our current version is 1.1.12 which will get incremented to 1.11.13



The image shows a code editor with a dark theme. The active tab is 'pom.xml (java-maven-app)'. The code is an XML file for a Maven project. The version is highlighted in blue. A lightbulb icon is visible next to the version tag.

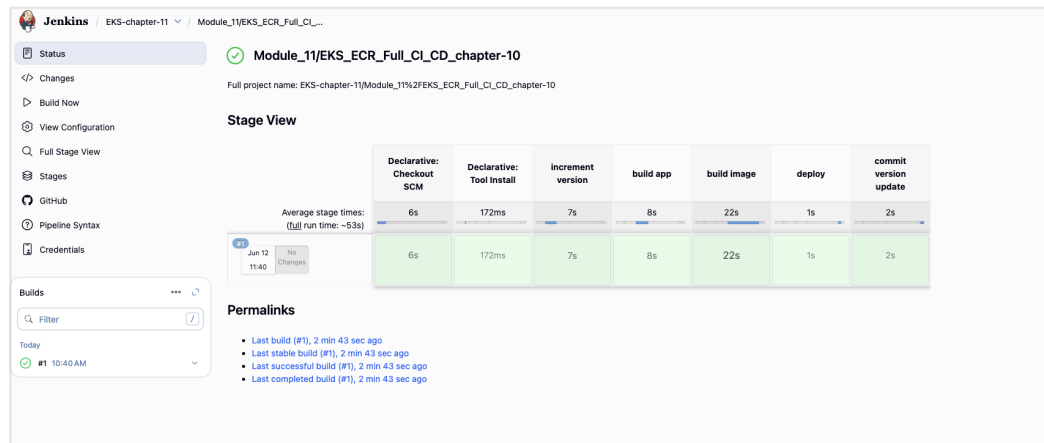
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         http://maven.apache.org/xsd/maven-4.0.0.xsd">
6
7     <groupId>com.example</groupId>
8     <artifactId>java-maven-app</artifactId>
9     <version>1.1.12</version>
10
11     <build>
12         <plugins>
```

And we didn't have pods running:

-> kubectl get pod

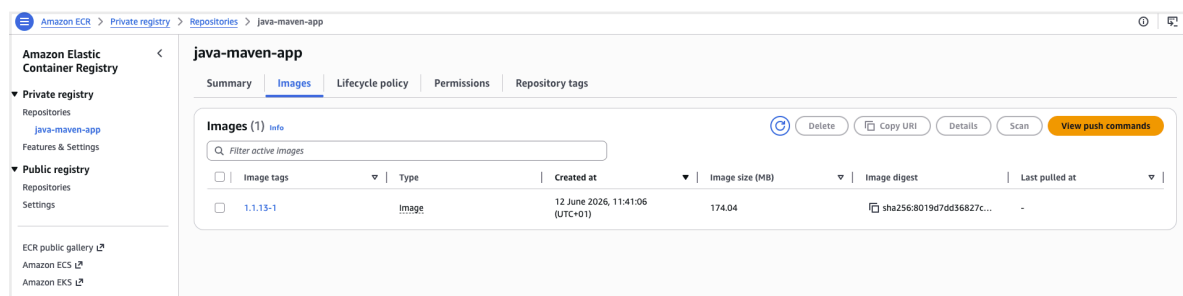
```
astrid@192.168.1.112 /Users/astrid
% kubectl get pod
No resources found in default namespace.
astrid@192.168.1.112 /Users/astrid
```

And the CI/CD pipeline job worked 🥳



Let's check our ECR repo -> and there is a new image in my repo with the version incremented to 1.11.13

The full tag is 1.11.13-1 where -1 stands for the number of the jenkins job build which in this case is One:



Let's check the cluster

-> kubectl get pod

And we can see that two pods were scheduled (however it looks like my EKS cluster needs change in config because they never change from pending to running, but for the purpose of this demo we can ignore this, also see notes: Module 11 - Extra: ECR using t2. Micro not enough to deploy java-maven-app)

```

astrid@192.168.1.112 /Users/astrid
% kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
java-maven-app-748f666ccf-fqxx2    0/1     Pending   0           5m30s
java-maven-app-748f666ccf-p9v9j    0/1     Pending   0           5m30s
astrid@192.168.1.112 /Users/astrid
% █

```

-> kubectl describe pod java-maven-app-748f666ccf-fqxx2

We can see that the image was pulled from ECR 🤖

```

astrid@192.168.1.112 /Users/astrid
% kubectl describe pod java-maven-app-748f666ccf-fqxx2
Name:                java-maven-app-748f666ccf-fqxx2
Namespace:            default
Priority:              0
Service Account:      default
Node:                 <none>
Labels:               app=java-maven-app
                     pod-template-hash=748f666ccf
Annotations:          <none>
Status:               Pending
IP:                   <none>
IPs:                  <none>
Controlled By:        ReplicaSet/java-maven-app-748f666ccf
Containers:
  java-maven-app:
    Image:              706976333940.dkr.ecr.eu-west-2.amazonaws.com/java-maven-app:1.1.13-1
    Port:               8080/TCP
    Host Port:          0/TCP
    Environment:        <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-d9lrb (ro)

```

NOTE

There are multiple ways of deploying from ECR to EKS

```

1 sh 'echo $PASS | docker login -u $USER --password-stdin ${DOCKER_REPO_SERVER}'
2 sh "docker push ${DOCKER_REPO}:${IMAGE_NAME}"
3 }
4 }
5 }
6 stage('deploy') {
7   environment {
8     AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
9     AWS_SECRET_ACCESS_KEY = credentials('jenkins_aws_secret_access_key')
10    APP_NAME = 'java-maven-app'

```

Multiple ways of deploying from ECR to EKS